Customer Purchasing Patterns Through Big Data Analytics and Machine Learning

Big Data for Better Decision Making

Ananya Geetey (25060242014)
Ayushi Sensharma (25060242031)



Problem Statement

- Large-scale data processing: Millions of grocery transactions make efficient data processing challenging.
- Business challenge: Identifying customer purchasing patterns and creating segments.

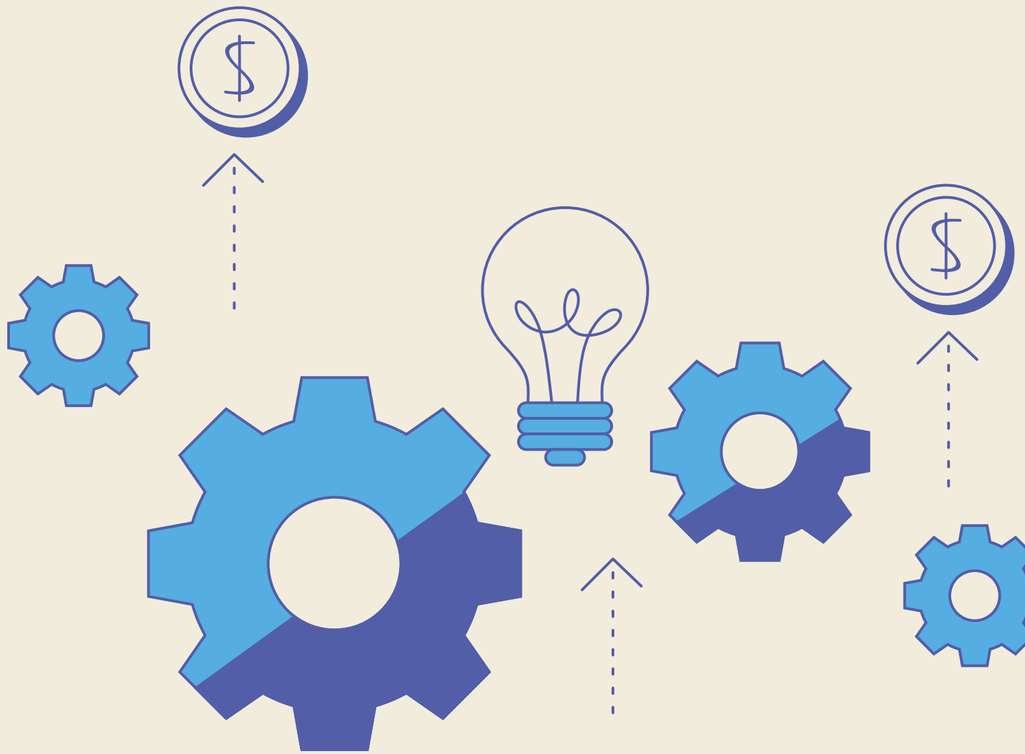


Objectives

Process large - scale transactional data using PySpark for efficient data processing

Performing exploratory data analysis using pandas, matplotlib and seaborn

Applying KMeans clustering using ScikitLearn



Research Questions

RQ1: How can PySpark be used to efficiently process large-scale grocery transaction data?

RQ2: What patterns exist in customer purchasing behaviour?

RQ3: How can K-Means identify and differentiate customer segments?

Data set Description

- Dataset Name: Supermarket Dataset for Predictive Marketing 2023
- Source: Kaggle
- Retail Chain: Hunter Supermarket, operating across 10 countries
- Dataset Size: 2,019,501 observations and 12 variables
- Data Type: Transactional data
- Unit of Observation: Individual product entry within an order

Variable	Meaning
Order ID	Unique identifier for each order placed by a customer.
User ID	Identifies individual customers and helps track their purchasing behaviour.
Product ID	Unique identifier for each product purchased.
Product Name	Name of the product purchased by the customer.
Department	Category or department to which the product belongs.
Department ID	Numerical identifier assigned to each product department.
Order Number	Indicates the sequence of an order in a customer's purchase history.
Day of Week	Indicates the day of the week on which the order was placed.
Hour of Day	Indicates the hour at which the order was placed.
Reorder Indicator	Indicates whether the customer has purchased the product previously.
Days Since Prior Order	Number of days since the customer's previous order.

completed data transformation in 0.0914 seconds

Velocity

Structured,12 variables to analyse diverse customer behaviours

Variety

Veracity

Data cleaning involved handling missing values removing whitespace from strings, and examining outliers

2,019,501 observations and 12 variables

Volume

Value

K-Means clustering was used to identify distinct customer segments

Big Data 5V Justification

Tools and Project Architecture

1

Data Source

Kaggle CSV

Supermarket dataset
2.02M rows

2

Big Data Processing

Apache Spark / PySpark

Load and inspect
Clean, check nulls

3

Feature Engineering

Pandas, NumPy

Customer features
Log transform

4

Exploratory Analysis

Matplotlib, Seaborn

Histograms, boxplots
Correlation heatmaps

5

Segmentation

Scikit-learn

Scale, then K-Means
k=4 chosen

Data Workflow

Data Cleaning → **Preprocessing** → **Feature Engineering** → **EDA**

- Whitespace removal from product name and department name
- Missing Values in ‘days_since_prior_order’

- Outliers in ‘add_to_card_order’

Data Workflow

Data Cleaning → **Preprocessing** → **Feature Engineering** → **EDA**

- Customer-Level Aggregation: Aggregated product-level transactions by user_id to create customer-level behavioural metrics.
- First-Time Customer Treatment: Separated customers with only one recorded order into a separate cluster.

- Log Transformation: for variables which were higly skewed
- Feature Scaling: Standardised the selected clustering features using StandardScaler before applying K-Means

Data Workflow

Data Cleaning → **Preprocessing** → **Feature Engineering** → **EDA**

- Total Items per Order
- Total Orders: unique orders per customer
- Total Products Purchased

- Reorder Rate: Mean of reorders by customer
- Average Basket Size
- Average Days Between Orders

PySpark Implementation

Using a spark dataframe over an RDD

- Structured Data
- Easy Column Access
- Simpler Operations
- Query Optimisation
- Better Suitability

Pandas vs Apache Spark Comparison

- Data Fits in RAM
- Distributed Computing Overhead
- Single-Machine Setup
- Action Execution Costs

PySpark Implementation

```
start = time.perf_counter()

spark_transformed = (
    adf.select("user_id", "order_id", "product_id", "reordered")
       .filter(f.col("reordered") == 1)
       .withColumn("reorder_flag", f.lit(1))
)

spark_transform_time = time.perf_counter() - start

print(f"Spark transformation-definition time: "
      f"{spark_transform_time:.6f} seconds")
```

```
Spark transformation-definition time: 0.091431 seconds
```

```
Spark:  0.091431 seconds
Pandas: 0.096776 seconds
Spark was faster at defining the transformations.
```

Pandas vs Apache Spark Comparison

- Data Fits in RAM
- Distributed Computing Overhead
- Single-Machine Setup

PySpark Implementation

- Spark Session Initialisation
- Dataset Loading
- Data Inspection
- Data Quality Checks
- Data Cleaning

```
file_path = "/content/drive/MyDrive/BDA/ECommerce_consumer behaviour.csv"
```

```
spark = SparkSession.builder.appName("ECommerce").getOrCreate()

adf = spark.read \
    .option("header", True) \
    .option("inferSchema", True) \
    .csv(file_path)
adf.printSchema()
```

```
root
 |-- order_id: integer (nullable = true)
 |-- user_id: integer (nullable = true)
 |-- order_number: integer (nullable = true)
 |-- order_dow: integer (nullable = true)
 |-- order_hour_of_day: integer (nullable = true)
 |-- days_since_prior_order: double (nullable = true)
 |-- product_id: integer (nullable = true)
 |-- add_to_cart_order: integer (nullable = true)
 |-- reordered: integer (nullable = true)
 |-- department_id: integer (nullable = true)
 |-- department: string (nullable = true)
 |-- product_name: string (nullable = true)
```

PySpark Implementation

```
adf = adf.withColumn(
    "total_items_per_order",
    f.count("product_id").over(Window.partitionBy("order_id"))
)

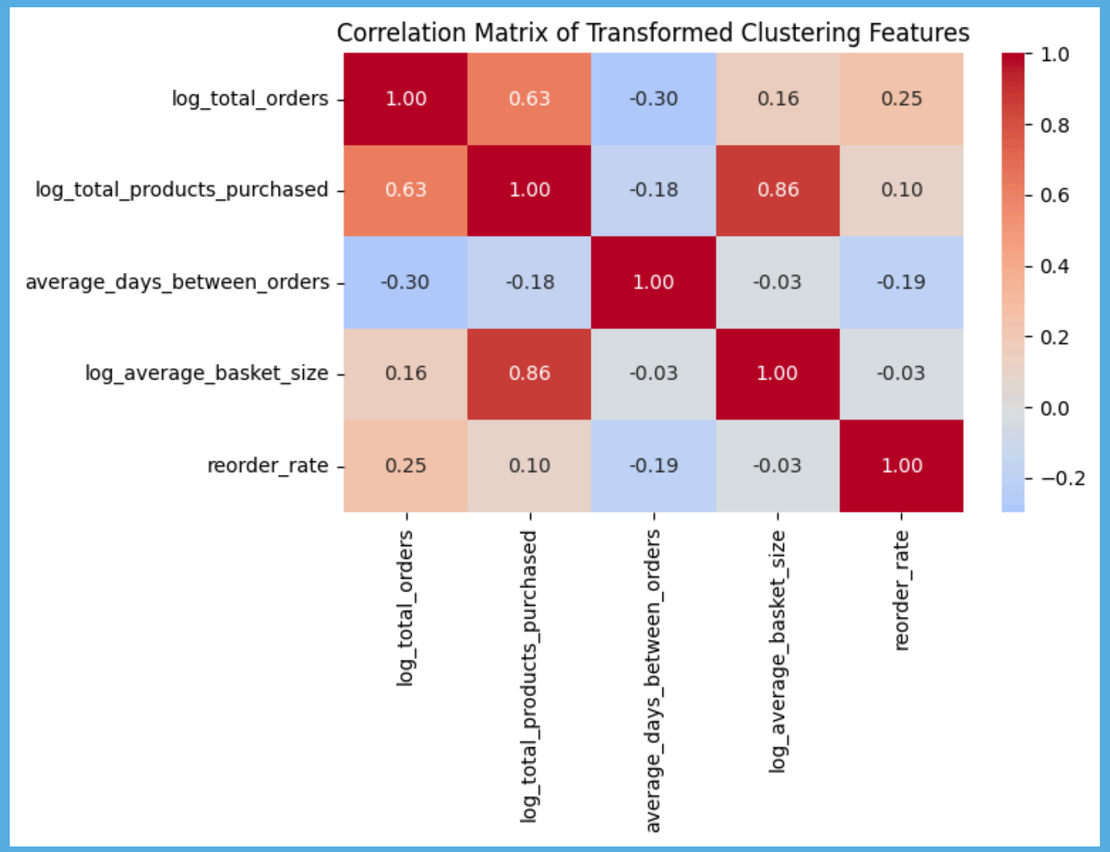
adf.select("order_id", "product_id", "total_items_per_order").show(5)
```

```
+--------+----------+---------------------+
|order_id|product_id|total_items_per_order|
+--------+----------+---------------------+
|      28|        36|                   16|
|      28|       108|                   16|
|      28|        35|                   16|
|      28|        24|                   16|
|      28|        84|                   16|
+--------+----------+---------------------+
only showing top 5 rows
```

```
df = adf.toPandas()
```

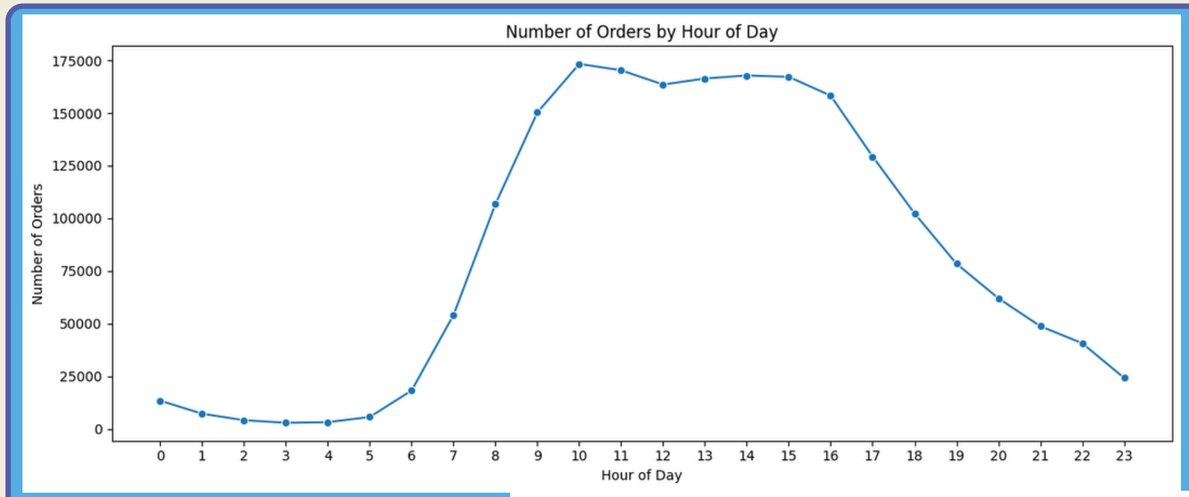
- Feature Engineering in PySpark
- Customer-Level Aggregation
- Pandas Conversion
- Exploratory Data Analysis
- Feature Preparation
- Customer Segmentation

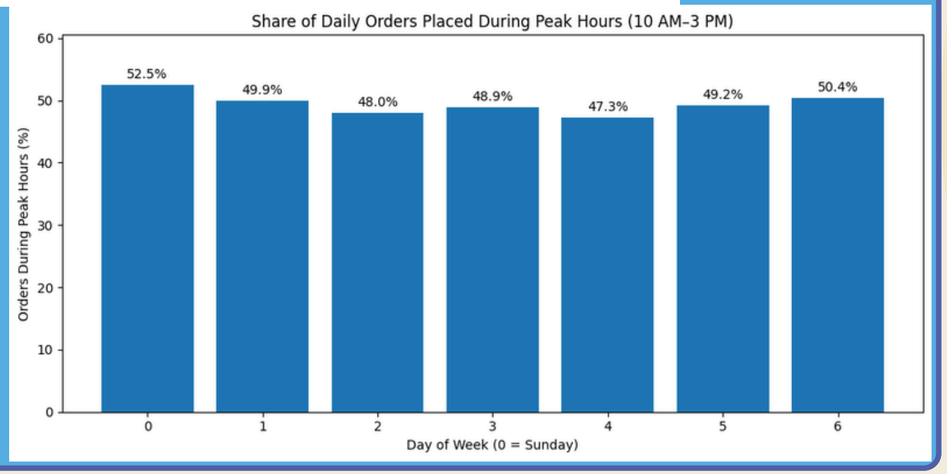
Exploratory Data Analysis



Correlation Analysis

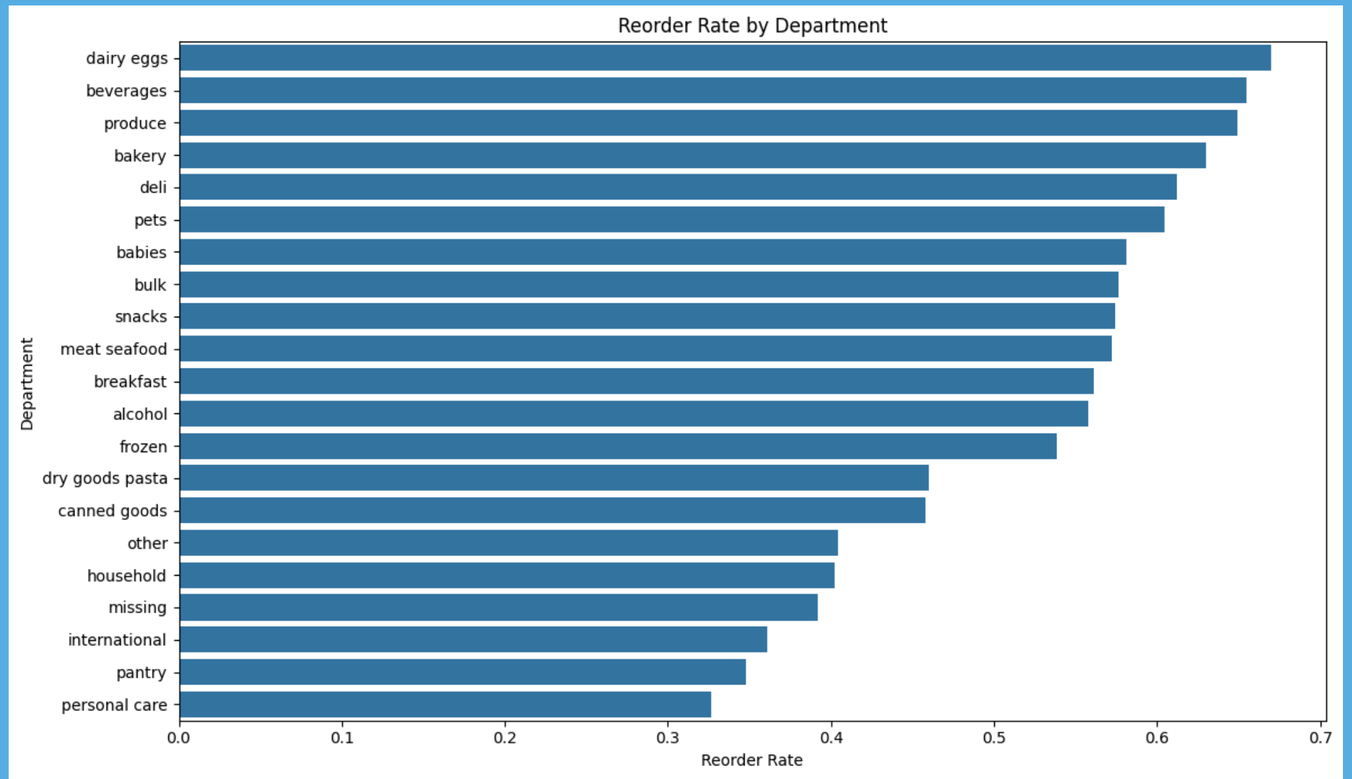
Exploratory Data Analysis





Temporal Purchasing Patterns

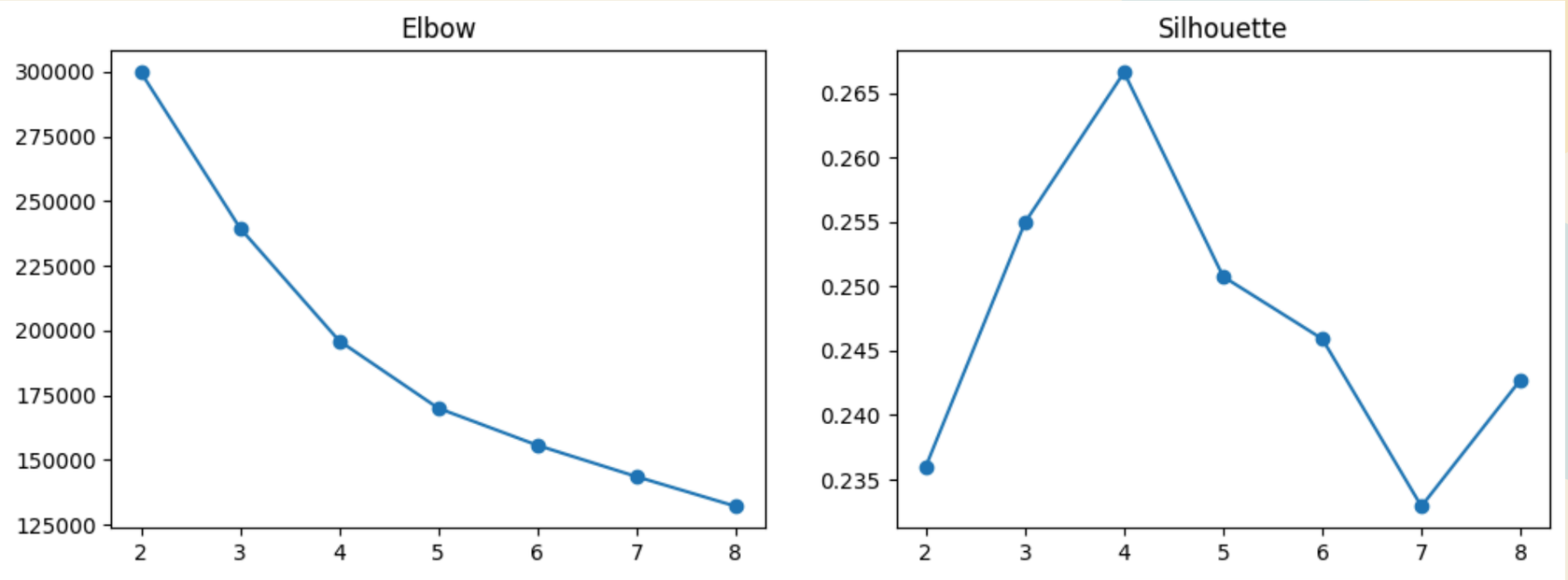
Exploratory Data Analysis



Department wise reorder

K Means Clustering

Selection of number of clusters



Elbow Method : showed a substantial reduction in inertia up to four clusters, after which the improvement diminished.

The Silhouette Score: highest at four clusters, with a value of approximately 0.267.

Therefore, we selected four clusters for the K-Means model

K Means Clustering

Selection of number of clusters

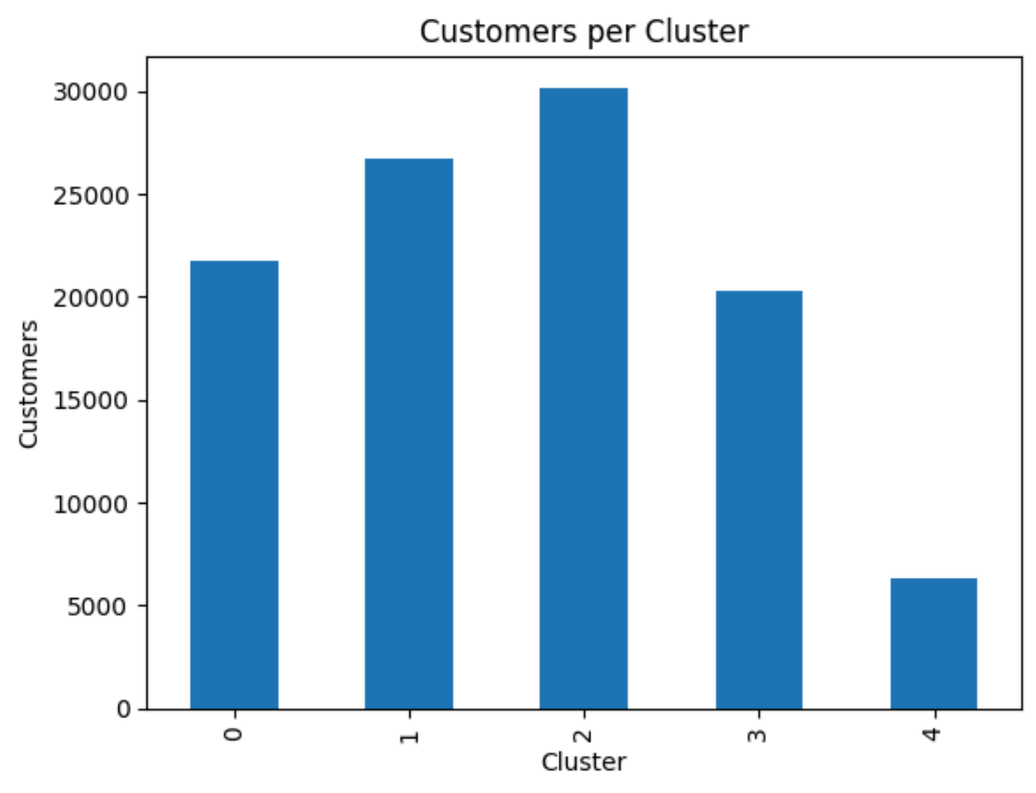
	0
user_id	0
total_orders	0
total_products_purchased	0
average_days_between_orders	6355
average_basket_size	0
reorder_rate	0

total_orders	
1	6355

- One additional group consisted of first-time customers identified separately.
- Average days between orders is one of the four clustering features.
- 6,355 customers had it missing, and all of the missing values came from first orders.
- A customer with only one order has no previous order to measure a gap from.
- K-Means is distance-based, so it cannot handle a missing value.
- Filling with the mean or median would invent a value that doesn’t describe first-time customers.
- Those artificial values would distort the distances and the other clusters.
- The final analysis therefore contains five customer groups

K Means Clustering

Distribution of customers across Clusters



Cluster	Customers	Customer Share (%)
0	21,772	20.68
1	26,725	25.39
2	30,144	28.63
3	20,277	19.26
4	6,355	6.04

K Means Clustering

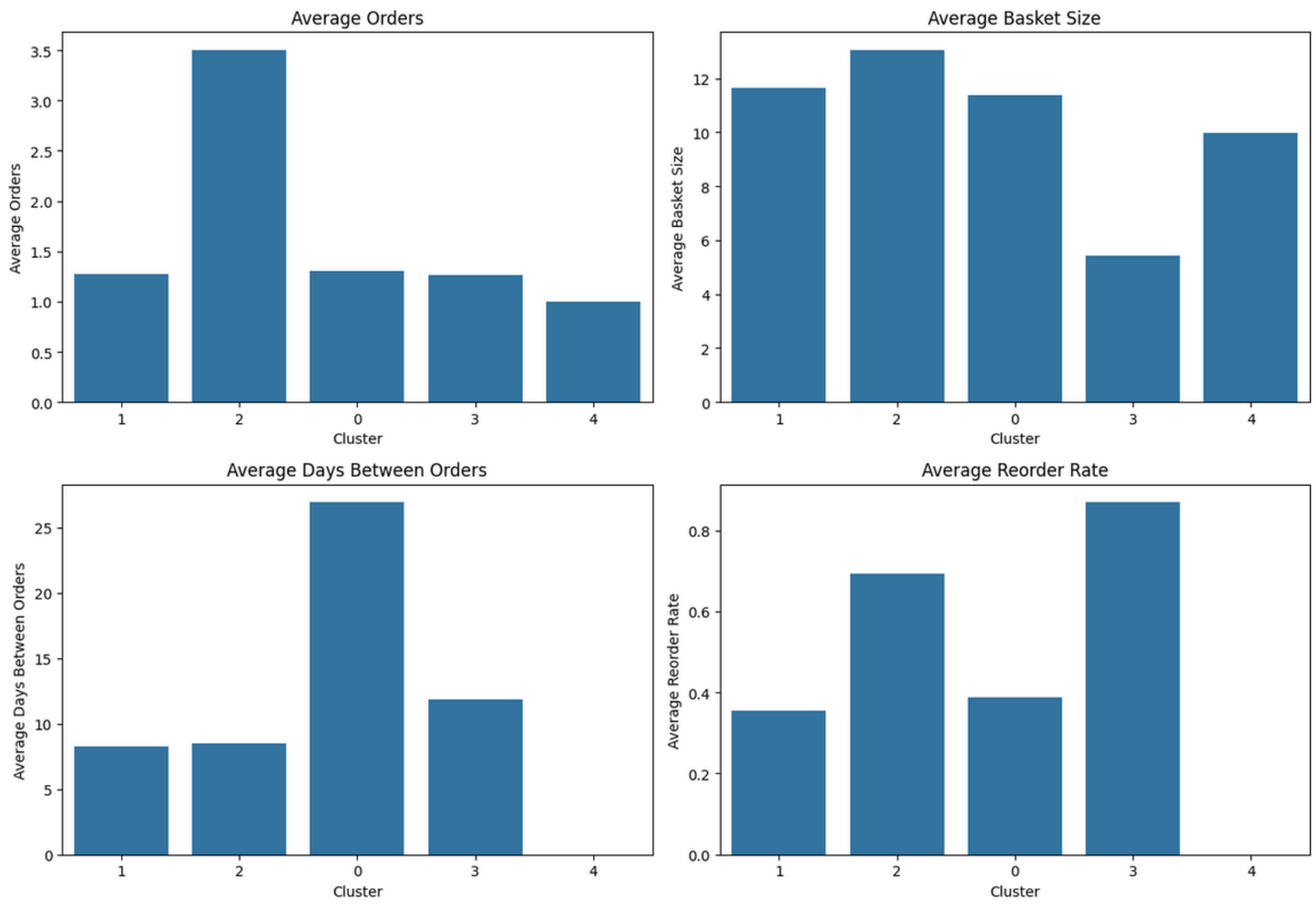
Cluster Profile

Cluster	Customers	Avg. Orders	Median Orders	Avg. Basket Size	Median Basket Size	Avg. Days Between Orders	Avg. Reorder Rate	Customer Share (%)
0	21,772	1.31	1	11.36	10	26.91	39%	20.68
1	26,725	1.27	1	11.63	10	8.23	36%	25.39
2	30,144	3.51	3	13.06	11.55	8.48	69%	28.63
3	20,277	1.26	1	5.43	4.14	11.84	87%	19.26
4	6,355	1	1	9.99	8	N/A	0%	6.04

- Cluster 0 : Occasional bulk buyers - few orders , longest gap (~27days) , fairly large baskets , low reorder rate (39%)
- Cluster 1 : Active explorers - frequent (~8days) with large baskets , but only 36% reorders
- Cluster 2 : Loyal customer - Most orders (3.5), biggest baskets (13), short gaps and 69% reorders: loyal, high value.
- Cluster 3: Small basket staple shoppers - Smallest baskets (5.4) but 87% reorders: buy the same staples
- Cluster 4 : First time customers - no reorder, no average day between order because they have placed order once only

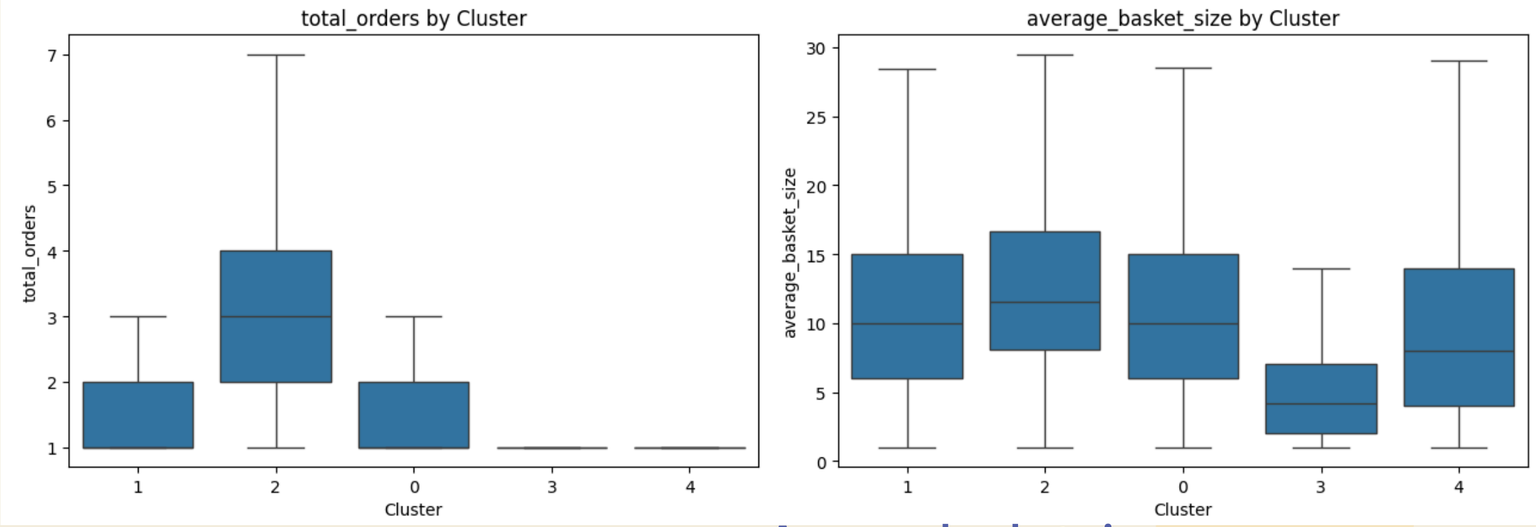
K Means Clustering

How clusters differ on each feature



K Means Clustering

How clusters differ on each feature



Total orders

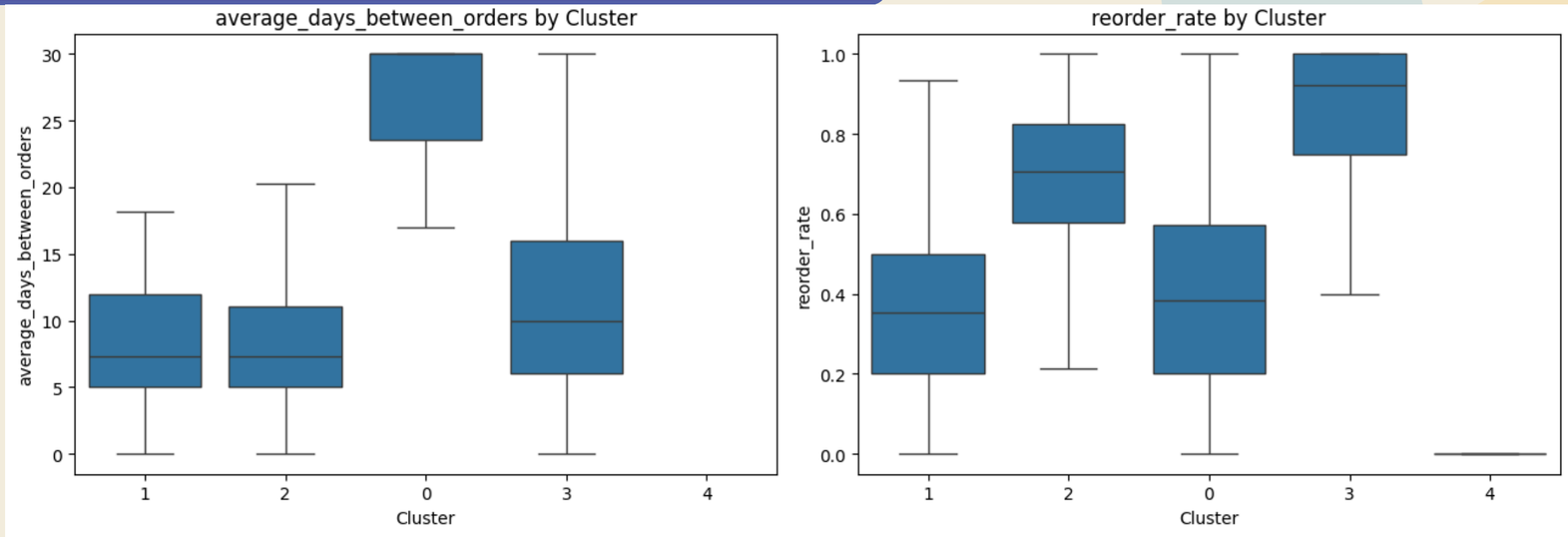
- Cluster 2 is clearly the most active: median 3 orders, middle 50% between 2 and 4, and some customers reach 7.
- Clusters 0 and 1 are mostly one-time or two-time buyers (median 1, IQR 1 to 2).
- Clusters 3 and 4 are flat at 1 order.

Average basket size

- Cluster 2 has the largest typical basket (median ≈ 11.5, IQR ≈ 8 to 16.5).
- Clusters 0 and 1 are almost identical (median ≈ 10, IQR ≈ 6 to 15).
- Cluster 3 has the smallest baskets (median ≈ 4, IQR ≈ 2 to 7).
- Cluster 4 sits in between (median ≈ 8) with wide spread.
- Basket size varies a lot within every cluster

K Means Clustering

How clusters differ on each feature



Average days between orders

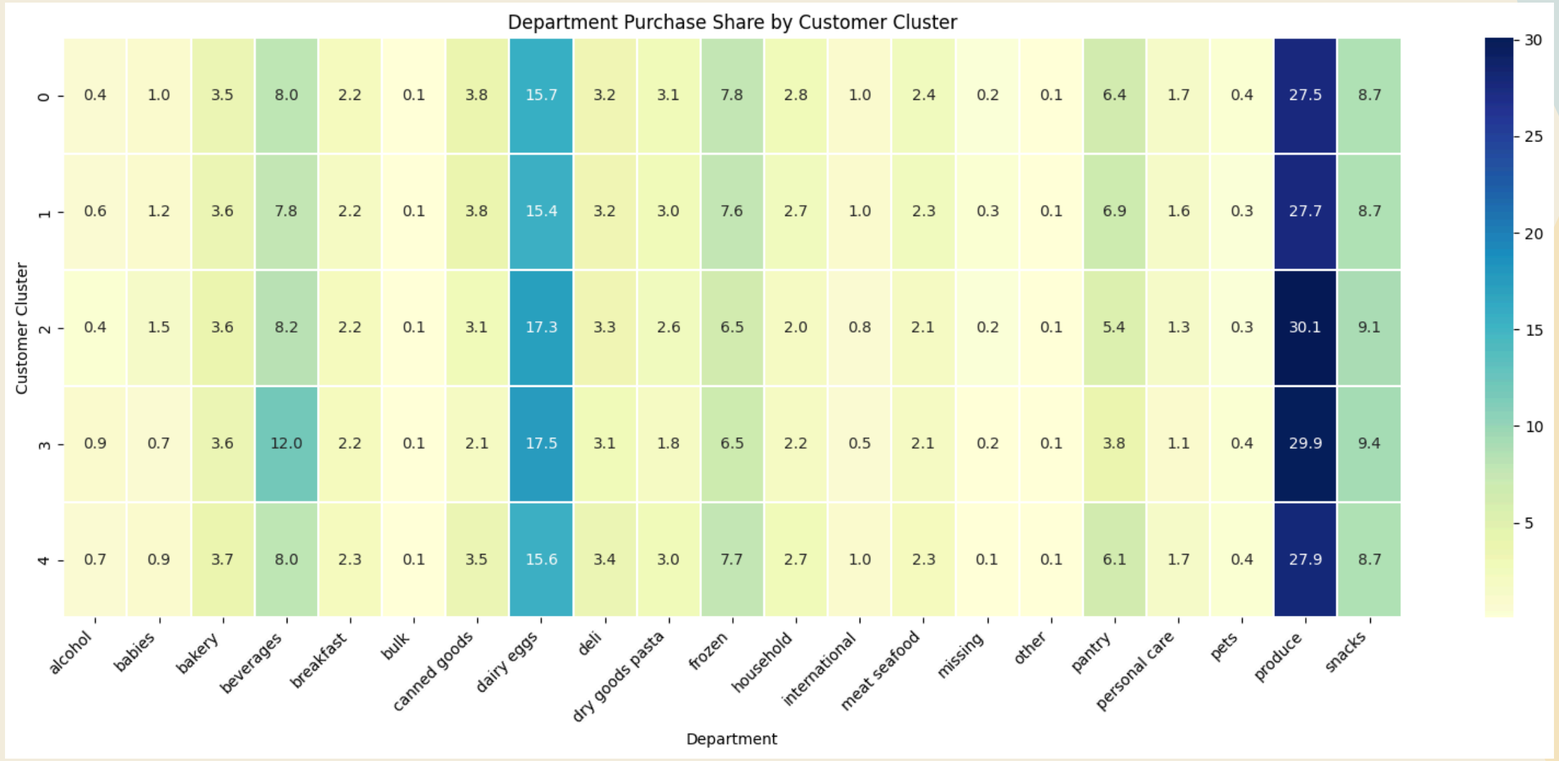
- Cluster 0 has the longest gaps. Its median is about 30 days, and even its lower whisker is around 17 days.
- Clusters 1 and 2 reorder fastest (median ≈ 7 days, IQR ≈ 5 to 12).
- Cluster 3 sits in the middle (median ≈ 10) but has the widest spread, from 0 to 30 days.
- Cluster 4 has no box because a gap needs at least two orders.

Reorder rate

- Cluster 3 is the most repeat-driven (median ≈ 0.92, IQR ≈ 0.75 to 1.0).
- Cluster 2 is next (median ≈ 0.71).
- Clusters 0 and 1 are low (medians ≈ 0.38 and 0.35).
- Cluster 4 is 0 by definition, since first orders can’t be reorders.

K Means Clustering

Department Wise Analysis



- Produce is the largest department in every cluster, at 27.5% (Cluster 0) to 30.1% (Cluster 2).
- Dairy and eggs is second in every cluster, at 15.4% (Cluster 1) to 17.5% (Cluster 3).
- Most departments are very consistent across clusters. Snacks, for example, range only from 8.7% to 9.4%
- But in certain departments clusters differ: beverages

K Means Clustering

Summary on clusters

CLUSTER 0
Occasional Bulk Buyers

FEATURES
- Few orders
- Longest gap Fairly large baskets
- Low reorder rate

RETAILER RESPONSE
Timed reminders around the 27-day gap

CLUSTER 1
Active Explorers

FEATURES
- Short gaps
- Fairly large baskets
- Low reorder rate
- Still trying products

RETAILER RESPONSE
Recommendations to turn trials into repeats

CLUSTER 2
Loyal Core

FEATURES
- Most orders
- Short gaps
- Biggest baskets
- Decent reorders

RETAILER RESPONSE
Protect with loyalty rewards

CLUSTER 3
Small Basket Staple Shoppers

FEATURES
- Few orders
- Decent gaps Smallest baskets
- Highest reorders

RETAILER RESPONSE
Cross-sell to grow small baskets

CLUSTER 4
First-Time Customers

FEATURES
- Exactly one order
- Interval is missing
- Decent basket size
- 0% reorders

RETAILER RESPONSE
Follow-up offers to earn a second order

Limitations

1. Transactional data only - no demographics, income, preferences or reasons for buying.
2. Outliers were kept - represent genuine bulk orders but can still influence the clusters, even after log transformation and standardisation.
3. Moderate cluster separation - silhouette score is about 0.267 at k = 4, so some segments overlap.
4. First-time customers handled separately
5. Future Scope : PySpark and its MLlib library to build an automated machine learning pipeline.

Conclusion

What we did

- Turned 2M+ grocery transaction records into customer segments
- Grouped customers by how they shop, not who they are

How we did it

- Cleaned and aggregated data with PySpark and Pandas
- Log-transformed and scaled four features
- K-Means with k = 4 (elbow and silhouette); first-timers set aside as Cluster 4

What we found

- 5 segments: Loyal Core (28.6%) orders most with the biggest baskets
- Staple Shoppers reorder 87% despite small baskets
- Produce and dairy lead everywhere, so segments differ in how they shop, not what they buy

Why it matters

- Retailers can tailor offers: loyalty rewards, cross-selling, timed reminders
- Clusters overlap a little (silhouette ≈ 0.267) and ideas are still untested

Thank You